

IoT/pwn mission room challenge

QUICK NOTE ON SHELL COMMANDS

\$ -> shell console on your attacker machine

-> console on the router device itself (via firmware emulation)

Setup a Docker environment

Tools installed: **binwalk**, **Ghidra** (GUI), **curl**, **wget**,

- Download the **Dockerfile** from the challenge site
- Run the following **Docker** commands

```
$ docker build -t firmware-analyzer .  
$ docker run -it --rm -v $(pwd):/workspace firmware-analyzer
```

NOTE: The commands in the Dockerfile are not part of the challenge itself

Useful commands cheatsheet

1. Binwalk

Firmware extraction tool (to access underlying filesystem)

- <https://github.com/refirmmlabs/binwalk>

Useful resources

1. <https://jarrettgxz-sec.gitbook.io/penetration-testing-ethical-hacking-concepts/iot-hardware-hacking/techniques/firmware/firmware-reversing-analysis#binwalk>

From within the Docker environment:

```
# Extract underlying filesystem  
$ binwalk --extract firmware.bin  
  
$ ls linksys_e1200.bin.extracted/squashfs-root  
bin dev etc home ...
```

2. FirmAE

Firmware emulation tool

- <https://github.com/pr0v3rbs/FirmAE>

Using a Docker environment:

```
git clone --recursive https://github.com/pr0v3rbs/FirmAE
cd FirmAE
./docker-init.sh

# Check emulation capabilities
$ ./docker-helper.py -ec auto firmware.bin
```

Please be patient, as this command may take awhile to complete

- The output should look something like this:

```
└─$ sudo ./run.sh -c auto ../firmware-mod-kit/fmk/new-firmware.bin
[sudo] password for jarrettgxz:
[*] ../firmware-mod-kit/fmk/new-firmware.bin emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 5
[MODE] check
mke2fs 1.47.0 (5-Feb-2023)
e2fsck 1.47.0 (5-Feb-2023)
[*] infer network start!!!
/home/jarrettgxz/sentctf/FirmAE/./scripts/makeNetwork.py:136: SyntaxWarning: invalid escape sequence '\['
  prog = re.compile(b"^\\[[^\\]]*\\] firmadyne: ")
/home/jarrettgxz/sentctf/FirmAE/./scripts/makeNetwork.py:153: SyntaxWarning: invalid escape sequence '\['
  b"^ioctl_SIOCSIFHWADDR\\[[^\\]]+\\]: dev:([^ ]+) mac:0x([0-9a-f]+) 0x([0-9a-f]+)"
/home/jarrettgxz/sentctf/FirmAE/./scripts/makeNetwork.py:178: SyntaxWarning: invalid escape sequence '\['
```

- You should get the following results after it completes

```
[+] Network reachable on 192.168.1.1!
[+] Web service on 192.168.1.1
[*] cleanup
```

Next, we can start the emulation and debug interface

```
# Run debug console and emulation
$ ./docker-helper.py -ed auto firmware.bin
```

- After starting the emulation, we will be able to access the device from a local address (eg. **192.168.1.1**)
- We can also drop ourselves into a debug console, by selecting option 2

```
[*] IP - 192.168.1.1
[*] connecting to netcat (192.168.1.1:31337)
[+] netcat connected
```

| FirmAE Debugger |

```
1. connect to socat
2. connect to shell
3. tcpdump
4. run gdbserver
5. file transfer
6. exit
> █
```

- This will provide us a shell access to the device itself, where we can perform further investigations

This option provides a shell console that mimics the actual Linux command-line environment that is present on the actual physical device

However, note that some commands may not exist, as IoT devices are usually stripped-down, and only provides a lightweight shell

- We can run basic Linux commands to interact with the filesystem, view running processes, etc.

```
# ps w
# netstat -tln
# ls /usr/sbin
```

If for whatever reason, you wish to install the **FirmAE** tool and related scripts manually, you can follow the steps below:

Requires **sasquatch**. This tool can be installed by default with the Docker setup step above (**binwalk** section)

```
# Install (**MOVE TO DOCKERFILE)
$ git clone https://github.com/pr0v3rbs/FirmAE.git

# Setup (**MOVE TO DOCKERFILE)
$ cd FirmAE
$ ./download.sh
$ ./install.sh
$ ./init.sh

# Check emulation capabilities
```

```
$ sudo ./run.sh -c auto firmware.bin
# Run debug console and emulation
sudo ./run.sh -d auto firmware.bin
```

3. Ghidra

Binary reversing + source code analysis

If you have ran the Docker setup commands, you can simply navigate to <http://localhost:8080> to access the Ghidra interface

If for whatever reason, you wish to install it manually, you may follow the steps below:

```
# Download from source + setup steps
sudo apt install wget
wget
https://github.com/NationalSecurityAgency/ghidra/releases/download/Ghidra_12.1.2_build/ghidra_12.1.2_PUBLIC_20260605.zip

sudo unzip ghidra_12.1.2_PUBLIC_20260605.zip -d /opt
sudo ln -s /opt/ghidra_12.1.2_PUBLIC/ghidraRun /usr/local/bin/ghidra

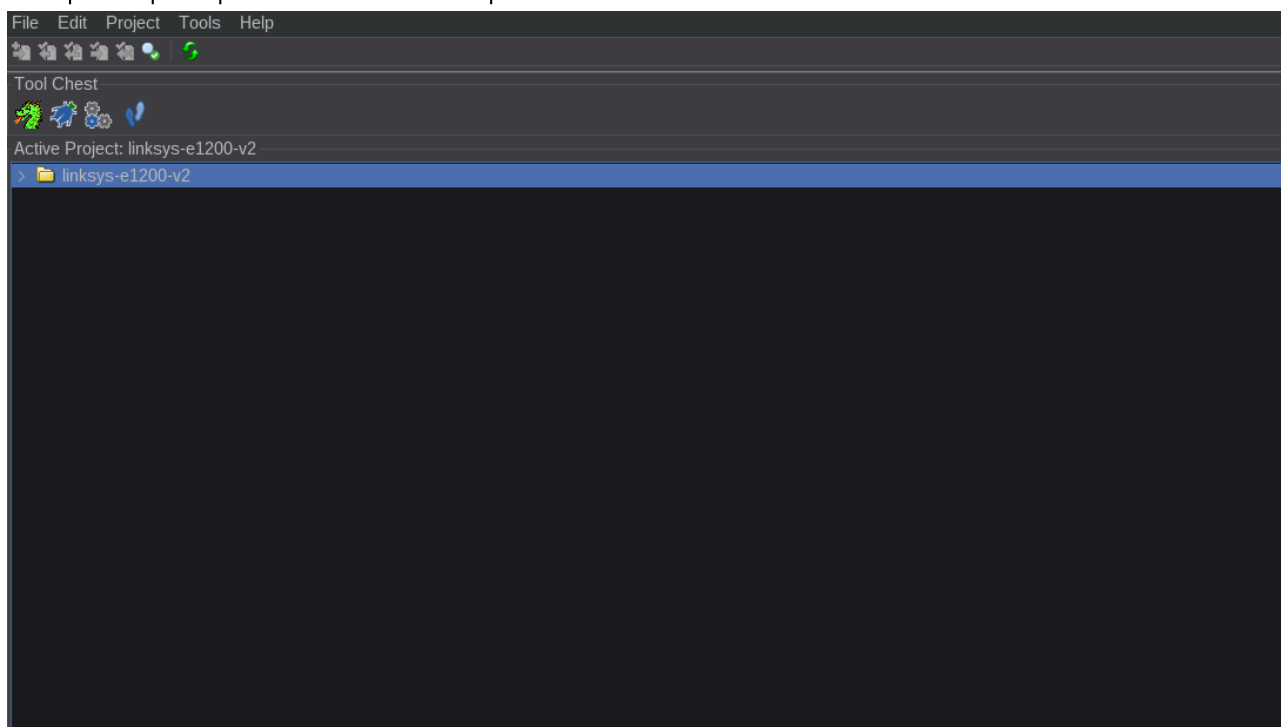
# Install dependencies
sudo apt install openjdk-X21-jdk

# Check that Ghidra is installed and start the application
which ghidra
ghidra
```

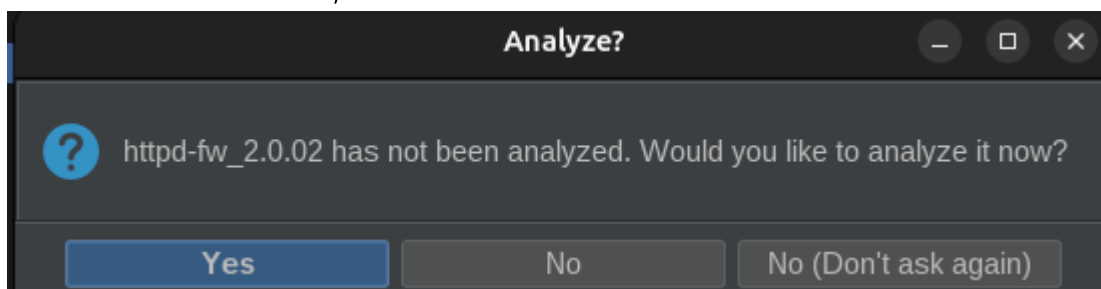
Reverse the vulnerable binary

1. Drag the file (or open using the options) into the Ghidra application

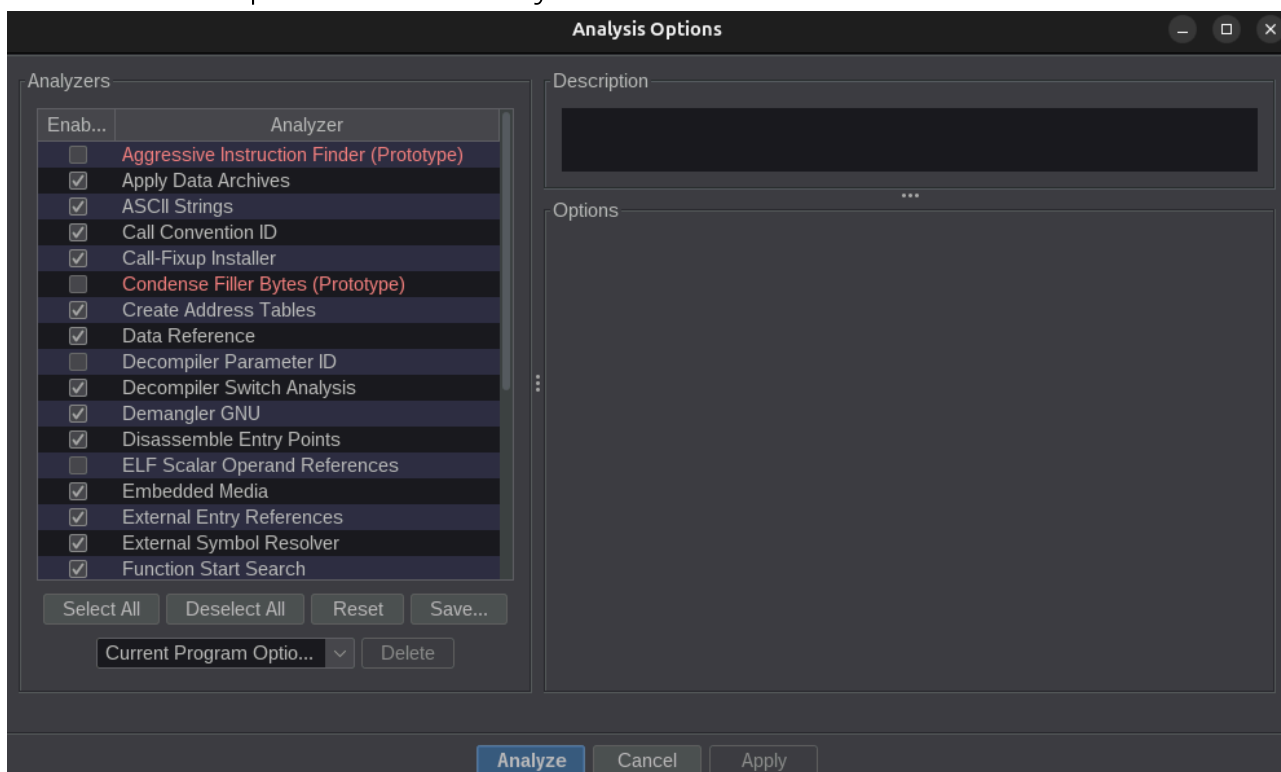
- Accept the prompts with the default options



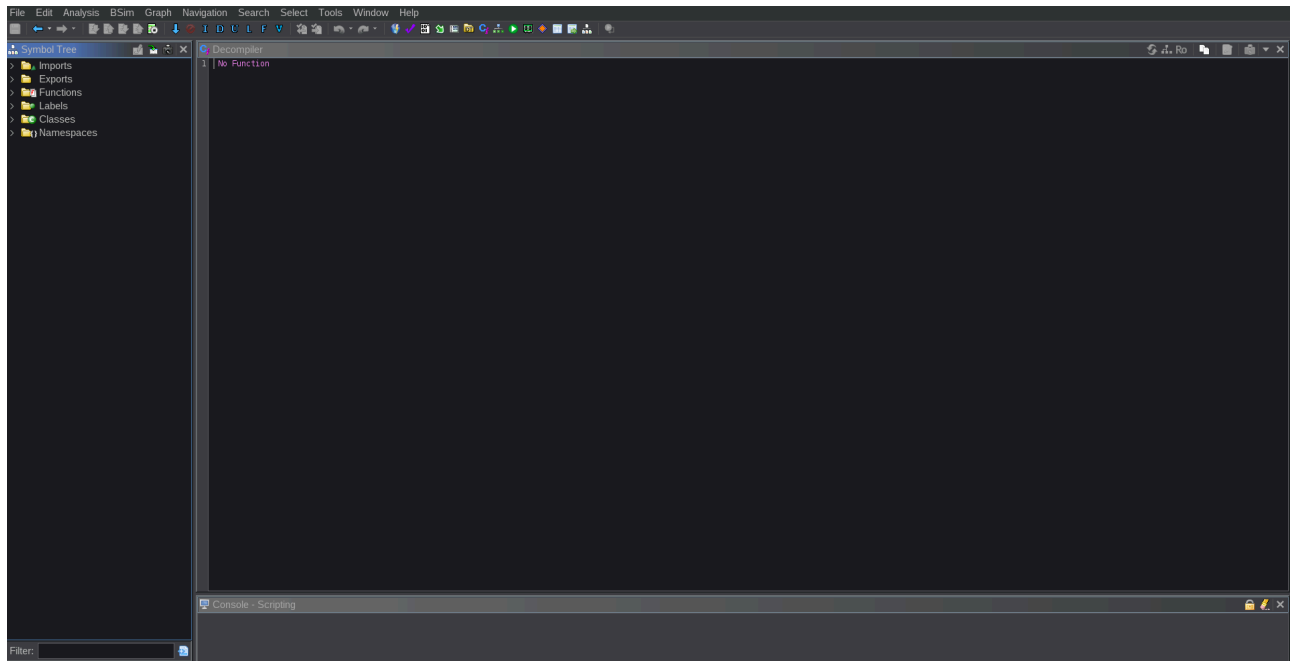
- Double-click the file name, and select "Yes"



- Leave the default options and select "Analyze"



4. Start analyzing the binary!



4. cURL, wget (exploit development+testing)

- **Command-line tools for sending HTTP requests**

```
# cURL eg.
curl -X POST http://192.168.1.1/PATH \
  --header "content-type: application/x-www-form-urlencoded" \
  --data "action=test&change_action=test"

# wget eg.
wget --method=POST http://192.168.1.1/PATH \
  --header "content-type: application/x-www-form-urlencoded" \
  --body-data "action=test&change_action=test"
```